Sistema de Controle de Planos de Operadora — Fase 2

1. Visão geral

Esta fase finaliza o sistema proposto integrando o ServicoGestao da Fase 1 com o ServicoFaturamento e o ServicoPlanosAtivos . O acesso externo é feito pelo api-gateway , enquanto a comunicação de eventos utiliza o message-broker HTTP incluído nesta entrega. O ServicoGestao , o ServicoFaturamento e o ServicoPlanosAtivos utilizam bancos MySQL próprios e isolados. Cada serviço é responsável pelas suas tabelas e seus dados; o cache do ServicoPlanosAtivos é apenas uma otimização sobre a projeção persistida.

2. Estrutura

amanda-desenvol-sistemas-backend-fase-2/

├── servico-gestao/

├── servico-faturamento/

├── servico-planos-ativos/

├── message-broker/

├── api-gateway/

├── amanda_Desenvolvimento_de_Sistemas_backend_Fase-2.postman_collection.json ├── Amanda_relatorio.pdf

├── MAPEAMENTO-FASE-2.md

└── README.md

O api-gateway escuta na porta 3000. O ServicoGestao escuta na 3001, o ServicoFaturamento na 4001, o ServicoPlanosAtivos na 4002 e o message-broker na 4000.

3. Pré-requisitos

É necessário Node.js 20 ou superior, npm e três bancos MySQL independentes. O projeto também contém um broker HTTP executável localmente, sem exigir instalação adicional de RabbitMQ.

4. Configuração do ServicoGestao

cd servico-gestao

cp .env.example .env

npm install

npm run seed

Preencha no .env as credenciais do banco do ServicoGestao . Os exemplos .env.example dos serviços da Fase 2 indicam as credenciais e portas dos seus respectivos bancos. O exemplo da Fase 2 usa PORT=3001 , para que a porta 3000 permaneça disponível ao gateway.

Os serviços possuem os seguintes bancos próprios:

Serviço	Banco	Tabelas principais
ServicoGestao	gestao	clientes , planos , assinaturas
ServicoFaturamento	faturamento	cobrancas , pagamentos
ServicoPlanosAtivos	planos_ativos	planos_ativos

As tabelas dos serviços da Fase 2 são criadas automaticamente na inicialização. O faturamento usa transação e chave única de idempotência para impedir pagamentos duplicados. O serviço de planos ativos usa chave primária por assinatura e atualização idempotente da projeção.

5. Instalação dos serviços

Execute a instalação dentro de cada pasta, em terminais separados ou sequencialmente:

cd message-broker & npm install

cd ./servico-faturamento & npm install

cd ./servico-planos-ativos & npm install

cd ./api-gateway & npm install

6. Execução

Antes de iniciar os serviços, crie os bancos gestao , faturamento e planos_ativos no MySQL. Execute os arquivos servico-gestao/database.sql , servico-faturamento/database.sql e servico-planos-ativos/database.sql com um usuário autorizado. As tabelas dos serviços da Fase 2 também são criadas automaticamente na primeira inicialização. Inicie primeiro os três bancos MySQL e depois, em terminais separados, execute:

cd message-broker & npm run dev

cd servico-faturamento & npm run dev

cd servico-planos-ativos & npm run dev

cd servico-gestao & npm run dev

cd api-gateway & npm run dev

O gateway estará disponível em http: /localhost:3000 . O endpoint GET /health do gateway verifica os quatro serviços. Para execução sem watch, substitua npm run dev por npm start.

7. Fluxo integrado de pagamento

Crie uma cobrança pelo gateway usando POST /faturamento/cobrancas com codigoAssinatura e valor . Em seguida, processe-a com POST /faturamento/cobrancas/:codigoCobranca/pagamentos . Quando o pagamento for aprovado, o ServicoFaturamento publica PagamentoAprovado ; o broker encaminha o evento ao ServicoGestao e ao ServicoPlanosAtivos . O primeiro atualiza dataUltimoPagamento , e o segundo atualiza sua projeção e suas consultas de planos ativos.

Para testar uma recusa, envie { "aprovar": false } . Para estornar um pagamento aprovado, utilize POST /faturamento/pagamentos/:codigoPagamento/estorno . Os consumidores tratam reentrega de evento sem duplicar a projeção.

8. Endpoints principais pelo gateway

Método	Endpoint	Serviço
GET	/health	Gateway e saúde dos serviços
GET	/gestao/clientes	ServicoGestao
GET	/gestao/planos	ServicoGestao
GET	/gestao/assinaturas/:tipo	ServicoGestao
POST	/faturamento/cobrancas	ServicoFaturamento
GET	/faturamento/cobrancas/:codigoCobranca	ServicoFaturamento
POST	/faturamento/cobrancas/:codigoCobranca/pagamentos	ServicoFaturamento
GET	/faturamento/pagamentos/:codigoPagamento	ServicoFaturamento
POST	/faturamento/pagamentos/:codigoPagamento/estorno	ServicoFaturamento
GET	/faturamento/assinaturas/:codigoAssinatura/pagamentos	ServicoFaturamento
GET	/planos-ativos	ServicoPlanosAtivos
GET	/planos-ativos/clientes/:codigoCliente	ServicoPlanosAtivos
GET	/planos-ativos/planos/:codigoPlano	ServicoPlanosAtivos

GET	/planos-ativos/assinaturas/:codigoAssinatura	ServicoPlanosAtivos

As rotas da Fase 1 continuam disponíveis sob /gestao e foram preservadas.

9. Postman

Importe amanda_Desenvolvimento_de_Sistemas_backend_Fase-2.postman_collection.json . A variável baseUrl está definida como http: /localhost:3000 , apontando para o gateway. A coleção contém health check, consultas da gestão, criação de cobrança, aprovação, recusa, estorno e consultas do ServicoPlanosAtivos .

10. Conclusão da fase

O principal desafio foi integrar serviços com responsabilidades independentes sem acoplar o ServicoGestao diretamente ao faturamento. A solução foi utilizar um gateway para as chamadas HTTP externas e um broker para eventos assíncronos. Também foi necessário garantir repetibilidade segura no processamento de pagamentos e nos consumidores, evitando duplicidade quando um evento for reenviado.

Na refação da Fase 1, a regra de ATIVO e CANCELADO foi ajustada para utilizar dataUltimoPagamento , deixando fimFidelidade apenas como informação do período de fidelidade. O mapeamento previamente solicitado da Fase 2 foi transformado em serviços executáveis, com contratos HTTP, eventos, projeção de planos ativos e integração com o gateway e o broker.